

INTRODUCCIÓN

En el presente documento se describe el desarrollo del sistema **EDD MedTrack**, el cual fue diseñado con el objetivo de aplicar los conocimientos adquiridos en el curso de Estructuras de Datos dentro de un entorno práctico.

El sistema permite la gestión de información hospitalaria, incluyendo personal médico, inventario de equipos, suministros, medicamentos y proveedores. Para ello, se implementaron diversas estructuras de datos como árboles AVL, árboles binarios de búsqueda (BST), árboles B, listas enlazadas y matrices dispersas.

El desarrollo del sistema se realizó utilizando el lenguaje de programación Perl junto con la biblioteca GTK3 para la interfaz gráfica, logrando una aplicación funcional que permite la interacción del usuario con las estructuras de datos de forma visual.

OBJETIVOS

2.1 Objetivo General

Desarrollar un sistema de gestión hospitalaria que implemente diferentes estructuras de datos para el almacenamiento, búsqueda y visualización eficiente de información.

2.2 Objetivos Específicos

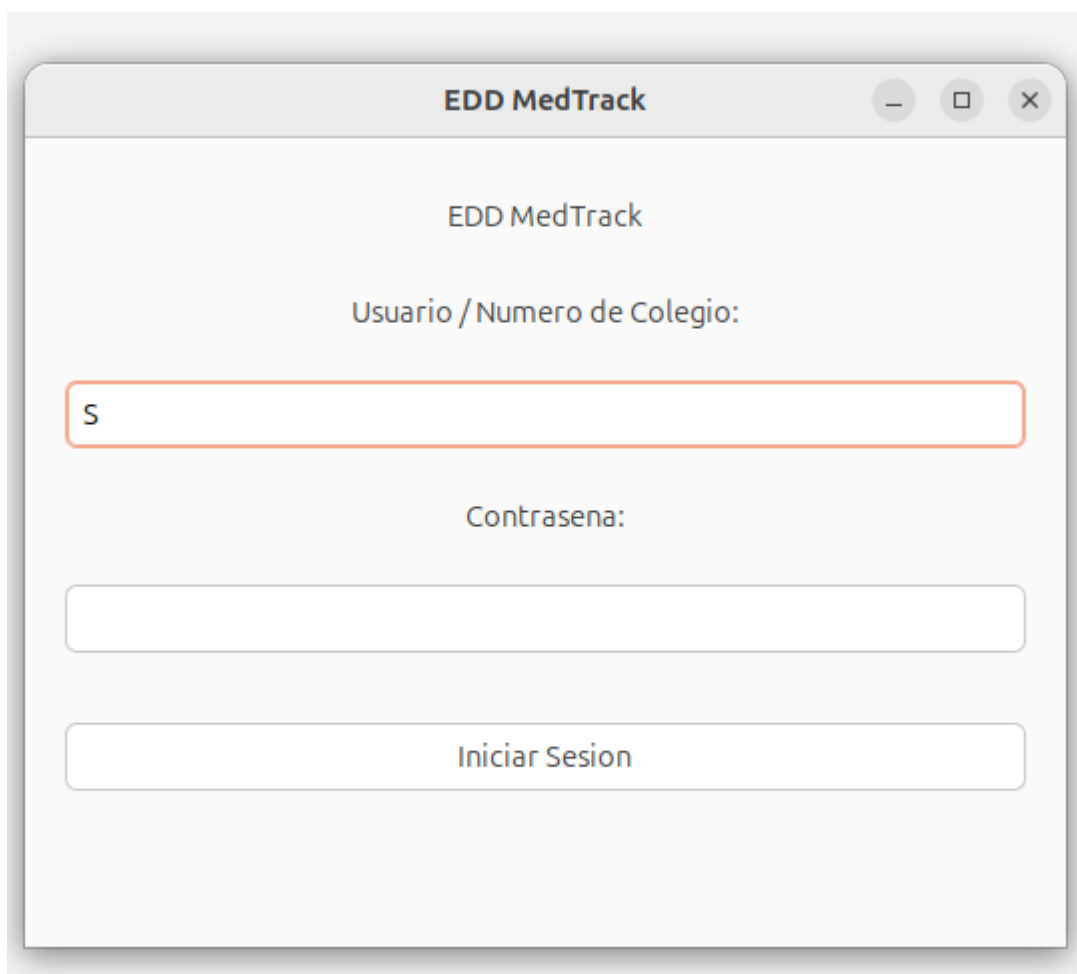
- Implementar un árbol AVL para la gestión del personal médico.
- Utilizar un árbol BST para el manejo del inventario de equipos médicos.
- Implementar un árbol B para la administración de suministros médicos.
- Utilizar una matriz dispersa para representar la relación entre proveedores y fabricantes.
- Desarrollar una interfaz gráfica amigable utilizando GTK3.
- Generar reportes visuales mediante Graphviz.

DESCRIPCIÓN GENERAL DEL SISTEMA

El sistema EDD MedTrack está diseñado como una aplicación modular, en donde cada tipo de información es gestionado mediante una estructura de datos específica.

El flujo general del sistema inicia con un proceso de autenticación, en el cual se validan las credenciales del usuario. Dependiendo del tipo de usuario, el sistema redirige a diferentes paneles con funcionalidades específicas.

El sistema permite realizar operaciones como inserción, eliminación, búsqueda, recorridos y generación de reportes gráficos.



The image shows a screenshot of a web application window titled "EDD MedTrack". The window has a light gray header bar with the title and standard window control buttons (minimize, maximize, close). The main content area is white and contains the following elements:

- The text "EDD MedTrack" centered at the top of the main area.
- The label "Usuario / Numero de Colegio:" centered below the title.
- A text input field with a red border containing the letter "S".
- The label "Contraseña:" centered below the first input field.
- An empty password input field.
- A button labeled "Iniciar Sesión" at the bottom.

ESTRUCTURAS DE DATOS IMPLEMENTADAS

4.1 Árbol AVL – Gestión de Personal

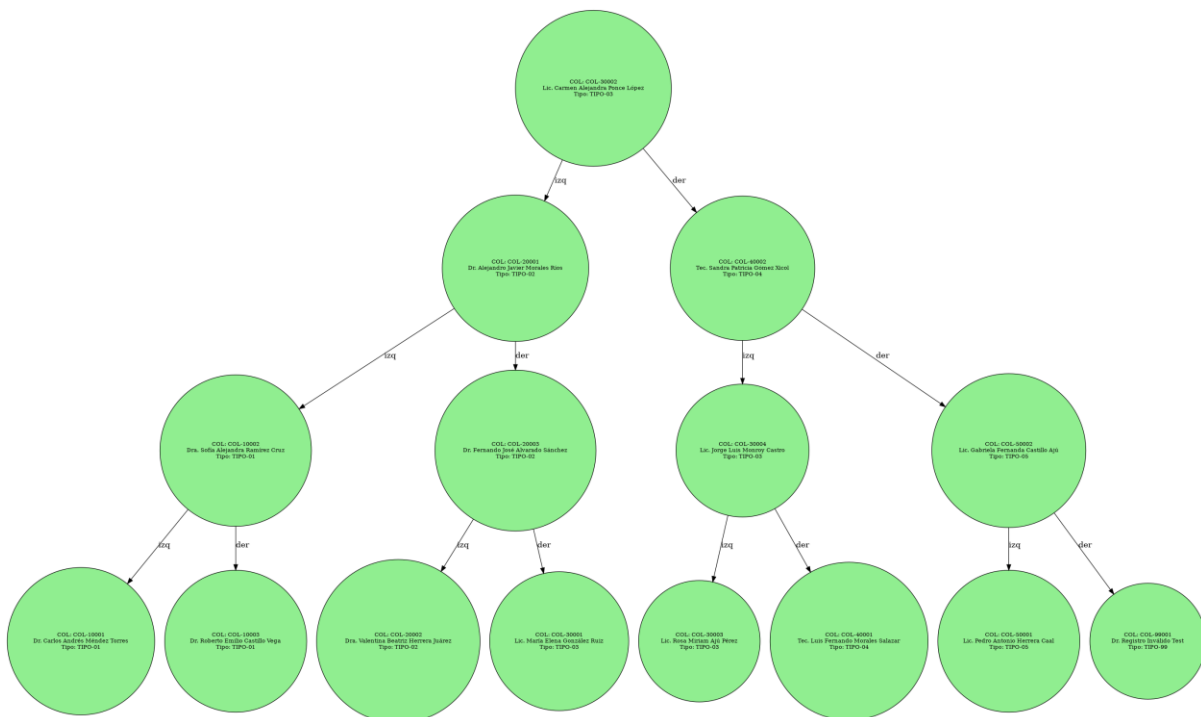
Para la gestión del personal médico se implementó un árbol AVL, el cual es un árbol binario de búsqueda auto-balanceado.

Esto permite mantener un tiempo de búsqueda eficiente, incluso cuando el número de registros es elevado.

Las operaciones implementadas en esta estructura incluyen:

- Inserción de usuarios
- Eliminación de usuarios
- Búsqueda por número de colegio
- Recorridos (PreOrden, InOrden y PostOrden)

El uso del árbol AVL garantiza que la altura del árbol se mantenga balanceada, evitando degradaciones en el rendimiento.



```

1 digraph AVL_Personal {
2   node [shape=circle, style=filled, fillcolor=lightgreen, fontsize=10];
3   rankdir=TB;
4
5   "COL-30002" [label="COL: COL-30002\nLic. Carmen Alejandra Ponce López\nTipo: TIPO-03"];
6   "COL-30002" -> "COL-20001" [label="izq"];
7   "COL-20001" [label="COL: COL-20001\nDr. Alejandro Javier Morales Ríos\nTipo: TIPO-02"];
8   "COL-20001" -> "COL-10002" [label="izq"];
9   "COL-10002" [label="COL: COL-10002\nDra. Sofía Alejandra Ramírez Cruz\nTipo: TIPO-01"];
10  "COL-10002" -> "COL-10001" [label="izq"];
11  "COL-10001" [label="COL: COL-10001\nDr. Carlos Andrés Múndez Torres\nTipo: TIPO-01"];
12  "COL-10001" -> "COL-10003" [label="der"];
13  "COL-10003" [label="COL: COL-10003\nDr. Roberto Emilio Castillo Vega\nTipo: TIPO-01"];
14  "COL-20001" -> "COL-20003" [label="der"];
15  "COL-20003" [label="COL: COL-20003\nDr. Fernando José Alvarado Sánchez\nTipo: TIPO-02"];
16  "COL-20003" -> "COL-20002" [label="izq"];
17  "COL-20002" [label="COL: COL-20002\nDra. Valentina Beatriz Herrera Juárez\nTipo: TIPO-02"];
18  "COL-20002" -> "COL-30001" [label="der"];
19  "COL-30001" [label="COL: COL-30001\nLic. María Elena González Ruiz\nTipo: TIPO-03"];
20  "COL-30001" -> "COL-40002" [label="der"];
21  "COL-40002" [label="COL: COL-40002\nTec. Sandra Patricia Gómez Xicol\nTipo: TIPO-04"];
22  "COL-40002" -> "COL-30004" [label="izq"];
23  "COL-30004" [label="COL: COL-30004\nLic. Jorge Luis Monroy Castro\nTipo: TIPO-03"];
24  "COL-30004" -> "COL-30003" [label="izq"];
25  "COL-30003" [label="COL: COL-30003\nLic. Rosa Miriam Ajó Pérez\nTipo: TIPO-03"];
26  "COL-30003" -> "COL-40001" [label="der"];
27  "COL-40001" [label="COL: COL-40001\nTec. Luis Fernando Morales Salazar\nTipo: TIPO-04"];
28  "COL-40001" -> "COL-50002" [label="der"];
29  "COL-50002" [label="COL: COL-50002\nLic. Gabriela Fernanda Castillo Ajó\nTipo: TIPO-05"];
30  "COL-50002" -> "COL-50001" [label="izq"];
31  "COL-50001" [label="COL: COL-50001\nLic. Pedro Antonio Herrera Caal\nTipo: TIPO-05"];
32  "COL-50001" -> "COL-99001" [label="der"];
33  "COL-99001" [label="COL: COL-99001\nDr. Registro Inválido Test\nTipo: TIPO-99"];

```

4.2 Árbol Binario de Búsqueda (BST) – Equipos Médicos

El árbol BST se utiliza para gestionar el inventario de equipos médicos.

Cada equipo es almacenado en el árbol utilizando su código como clave, lo que permite realizar búsquedas eficientes.

Se implementaron las siguientes operaciones:

- Inserción de equipos
- Eliminación
- Búsqueda por código
- Recorridos (PreOrden, InOrden y PostOrden)

El recorrido InOrden permite visualizar los equipos ordenados de forma ascendente según su código.

Visualizar inventario por medicamentos - BST

Codigo	Nombre	Fabricante	Precio	Cantidad	Fecha Ingreso	Nivel Minimo
EQUEQU-001	Monitor de Signos Vitales	MediTechGT	4500	10	2026-03-10	2
EQUEQU-002	Electrocardiógrafo 12 Derivaciones	MediTechGT	6200	5	2026-03-10	1
EQUEQU-003	Bomba de Infusión Volumétrica	BuenaVidaGT	1800	20	2026-03-10	5
EQUEQU-004	Tensiómetro Manual Aneroide	BuenaVidaGT	180	20	2026-03-12	4

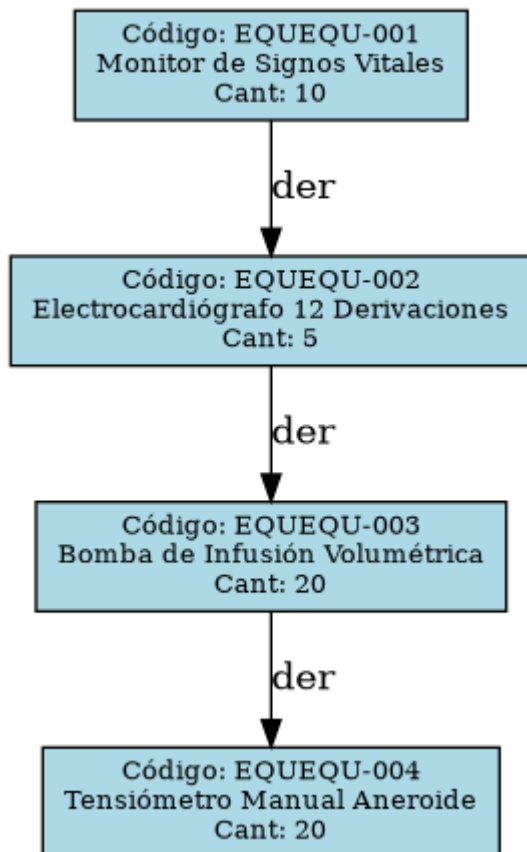
4.3 Árbol B – Suministros Médicos

Para el manejo de suministros médicos se implementó un árbol B de orden 4.

Este tipo de estructura es especialmente útil cuando se manejan grandes cantidades de datos, ya que permite mantener el árbol balanceado y reducir la cantidad de accesos necesarios para encontrar información.

Las operaciones implementadas incluyen:

- Inserción
- Búsqueda
- Eliminación
- Recorrido InOrden



4.4 Lista Doble Enlazada – Medicamentos

Los medicamentos son almacenados en una lista doblemente enlazada, lo que permite recorrer la lista en ambas direcciones.

Esto facilita operaciones como inserción y recorrido.

```
#!/usr/bin/perl
use strict;
use warnings;

package ListaDobleEnlazada;

sub new {
    my $class = shift;
    my $self = {
        cabeza => undef,
        cola => undef,
        tamano => 0
    };
    bless $self, $class;
```

```
return $self;
}

sub insertar_ordenado {
my ($self, $medicamento) = @_;

if (!defined $self->{cabeza}) {
$self->{cabeza} = $medicamento;
$self->{cola} = $medicamento;
$self->{tamano} = 1;
return;
}

my $actual = $self->{cabeza};
my $prev = undef;

while (defined $actual) {
if ($medicamento->obtener_codigo lt $actual->obtener_codigo) {
last;
}
$prev = $actual;
$actual = $actual->{siguiente};
}

if (!defined $prev) {
# insertar al inicio
$medicamento->{siguiente} = $self->{cabeza};
$self->{cabeza}->{anterior} = $medicamento;
$self->{cabeza} = $medicamento;
}
elsif (!defined $actual) {
# insertar al final
$self->{cola}->{siguiente} = $medicamento;
$medicamento->{anterior} = $self->{cola};
$self->{cola} = $medicamento;
}
else {
# insertar en medio
$prev->{siguiente} = $medicamento;
$medicamento->{anterior} = $prev;
$medicamento->{siguiente} = $actual;
$actual->{anterior} = $medicamento;
}

$self->{tamano}++;
}
```



```

sub mostrar_todos {
my $self = shift;

# ←←← ESTA LÍNEA ELIMINA LOS WARNINGS
return if !defined $self->{cabeza};

print "=== INVENTARIO (Lista Doblemente Enlazada) ===\n";
print "Codigo | Nombre | Cantidad | Fecha Venc. | Estado\n";
print "-----\n";

my $actual = $self->{cabeza};
while (defined $actual) {
my $estado = ($actual->obtener_cantidad < $actual->obtener_nivel_minimo)
? "BAJO STOCK"
: "NORMAL";

printf "%-8s | %-20s | %-8s | %-12s | %s\n",
$actual->obtener_codigo,
$actual->obtener_nombre,
$actual->obtener_cantidad,
$actual->obtener_fecha_vencimiento,
$estado;

$actual = $actual->{siguiente};
}

print "Total de medicamentos: " . $self->{tamano} . "\n\n";
}

1;

```

4.5 Lista Circular Doble – Proveedores

Los proveedores se gestionan mediante una lista circular doble, lo cual permite recorrer todos los elementos sin un inicio o fin definido.

```

#!/usr/bin/perl
use strict;
use warnings;

package ListaCircularDobleEnlazada;

sub new {

```

```
my $class = shift;
my $self = {
  cabeza => undef,
  tamano => 0
};
bless $self, $class;
return $self;
}

sub insertar {
  my ($self, $solicitud) = @_;
  if (!defined $self->{cabeza}) {
    $self->{cabeza} = $solicitud;
    $solicitud->{siguiente} = $solicitud;
    $solicitud->{anterior} = $solicitud;
    $self->{tamano} = 1;
    return;
  }
  my $ultimo = $self->{cabeza}->{anterior};
  $ultimo->{siguiente} = $solicitud;
  $solicitud->{anterior} = $ultimo;
  $solicitud->{siguiente} = $self->{cabeza};
  $self->{cabeza}->{anterior} = $solicitud;
  $self->{tamano}++;
}

sub procesar_primera {
  my $self = shift;
  return undef unless defined $self->{cabeza};
  return $self->{cabeza};
}

sub eliminar_primera {
  my $self = shift;
  return unless defined $self->{cabeza};
  if ($self->{tamano} == 1) {
    $self->{cabeza} = undef;
    $self->{tamano} = 0;
    return;
  }
  my $primera = $self->{cabeza};
  my $ultima = $primera->{anterior};
  $self->{cabeza} = $primera->{siguiente};
  $self->{cabeza}->{anterior} = $ultima;
  $ultima->{siguiente} = $self->{cabeza};
  $self->{tamano}--;
}
```

```

sub mostrar_todos {
my $self = shift;
return if !defined $self->{cabeza};
print "=== SOLICITUDES PENDIENTES ===\n";
my $actual = $self->{cabeza};
do {
print "No. " . $actual->obtener_numero . " | Dept: " . $actual->obtener_departamento .
" | Med: " . $actual->obtener_codigo_medimento .
" | Cant: " . $actual->obtener_cantidad .
" | Estado: " . $actual->obtener_estado . "\n";
$actual = $actual->{siguiente};
} while ($actual != $self->{cabeza});
print "Total pendientes: " . $self->{tamano} . "\n\n";
}

1;

```

4.6 Matriz Dispersa – Relación Proveedor-Fabricante

Se implementó una matriz dispersa para representar la relación entre proveedores y fabricantes.

Esta estructura permite optimizar el uso de memoria al almacenar únicamente las posiciones donde existe información.

```

#!/usr/bin/perl
use strict;
use warnings;

package MatrizDispersaProveedorFabricante;

use NodoCabecera; # ← Tu archivo original
use NodoDato; # ← Tu archivo original

# =====
# CONSTRUCTOR
# =====

sub new {
my $class = shift;
my $self = {
lista_filas => undef, # Cabeceras de PROVEEDORES
lista_cols => undef, # Cabeceras de FABRICANTES
num_filas => 0,
num_cols => 0,

```

```

total_datos => 0,
};
bless $self, $class;
return $self;
}

# =====
# BUSQUEDA DE CABECERAS
# =====

sub _buscar_cab_fila {
my ($self, $fila_idx) = @_;
my $actual = $self->{lista_filas};
while (defined $actual) {
return $actual if ($actual->get_label() == $fila_idx);
last if ($actual->get_label() > $fila_idx);
$actual = $actual->get_next();
}
return undef;
}

sub _buscar_cab_col {
my ($self, $col_idx) = @_;
my $actual = $self->{lista_cols};
while (defined $actual) {
return $actual if ($actual->get_label() == $col_idx);
last if ($actual->get_label() > $col_idx);
$actual = $actual->get_next();
}
return undef;
}

# =====
# CREAR CABECERAS
# =====

sub _obtener_o_crear_cab_fila {
my ($self, $fila_idx) = @_;
# ... (código idéntico al tuyo)
if (!defined $self->{lista_filas}) {
my $nueva = NodoCabecera->new($fila_idx);
$self->{lista_filas} = $nueva;
return $nueva;
}
if ($self->{lista_filas}->get_label() > $fila_idx) {
my $nueva = NodoCabecera->new($fila_idx);
$nueva->set_next($self->{lista_filas});
$self->{lista_filas} = $nueva;
return $nueva;
}

```

```

}
if ($self->{lista_filas}->get_label() == $fila_idx) {
return $self->{lista_filas};
}

my $anterior = $self->{lista_filas};
my $actual = $anterior->get_next();

while (defined $actual) {
if ($actual->get_label() == $fila_idx) {
return $actual;
}
if ($actual->get_label() > $fila_idx) {
my $nueva = NodoCabecera->new($fila_idx);
$nueva->set_next($actual);
$anterior->set_next($nueva);
return $nueva;
}
$anterior = $actual;
$actual = $actual->get_next();
}

my $nueva = NodoCabecera->new($fila_idx);
$anterior->set_next($nueva);
return $nueva;
}

sub _obtener_o_crear_cab_col {
my ($self, $col_idx) = @_;
# ... (código idéntico al tuyo, solo cambiado el nombre)
if (!defined $self->{lista_cols}) {
my $nueva = NodoCabecera->new($col_idx);
$self->{lista_cols} = $nueva;
return $nueva;
}
if ($self->{lista_cols}->get_label() > $col_idx) {
my $nueva = NodoCabecera->new($col_idx);
$nueva->set_next($self->{lista_cols});
$self->{lista_cols} = $nueva;
return $nueva;
}
if ($self->{lista_cols}->get_label() == $col_idx) {
return $self->{lista_cols};
}

my $anterior = $self->{lista_cols};

```

```

my $actual = $anterior->get_next();

while (defined $actual) {
if ($actual->get_label() == $col_idx) {
return $actual;
}
if ($actual->get_label() > $col_idx) {
my $nueva = NodoCabecera->new($col_idx);
$nueva->set_next($actual);
$anterior->set_next($nueva);
return $nueva;
}
$anterior = $actual;
$actual = $actual->get_next();
}

my $nueva = NodoCabecera->new($col_idx);
$anterior->set_next($nueva);
return $nueva;
}

# =====
# INSERTAR (Proveedor vs Fabricante)
# =====
sub insertar {
my ($self, $proveedor_nit, $fabricante, $cantidad) = @_;

# Mapear strings a índices dinámicos
my $fila_idx = _obtener_indice_fila($self, $proveedor_nit);
my $col_idx = _obtener_indice_col($self, $fabricante);

my $cab_fila = $self->_obtener_o_crear_cab_fila($fila_idx);
my $cab_col = $self->_obtener_o_crear_cab_col($col_idx);

# Verificar si ya existe
my $existente = $self->obtener($fila_idx, $col_idx);
if (defined $existente) {
my $valor_actual = $existente->get_valor();
$existente->set_valor($valor_actual + $cantidad); # acumular cantidad
return;
}

my $nuevo = NodoDato->new($fila_idx, $col_idx, $cantidad);

# Insertar en fila (horizontal)
if (!defined $cab_fila->get_right()) {

```

```

$cab_fila->set_right($nuevo);
} elseif ($cab_fila->get_right()->get_col() > $col_idx) {
my $primero = $cab_fila->get_right();
$nuevo->set_right($primero);
$primero->set_left($nuevo);
$cab_fila->set_right($nuevo);
} else {
my $anterior = $cab_fila->get_right();
my $actual = $anterior->get_right();
while (defined $actual && $actual->get_col() < $col_idx) {
$anterior = $actual;
$actual = $actual->get_right();
}
$nuevo->set_right($actual);
$nuevo->set_left($anterior);
$anterior->set_right($nuevo);
$actual->set_left($nuevo) if defined $actual;
}

# Insertar en columna (vertical)
if (!defined $cab_col->get_down()) {
$cab_col->set_down($nuevo);
} elseif ($cab_col->get_down()->get_fila() > $fila_idx) {
my $primero = $cab_col->get_down();
$nuevo->set_down($primero);
$primero->set_up($nuevo);
$cab_col->set_down($nuevo);
} else {
my $anterior = $cab_col->get_down();
my $actual = $anterior->get_down();
while (defined $actual && $actual->get_fila() < $fila_idx) {
$anterior = $actual;
$actual = $actual->get_down();
}
$nuevo->set_down($actual);
$nuevo->set_up($anterior);
$anterior->set_down($nuevo);
$actual->set_up($nuevo) if defined $actual;
}

$self->{total_datos}++;
}

# Helpers para mapear strings a índices
my %map_proveedores;
my %map_fabricantes;
my $fila_counter = 0;

```

```

my $col_counter = 0;

sub _obtener_indice_fila {
my ($self, $nit) = @_;
unless (exists $map_proveedores{$nit}) {
$map_proveedores{$nit} = $fila_counter++;
}
return $map_proveedores{$nit};
}

sub _obtener_indice_col {
my ($self, $fabricante) = @_;
unless (exists $map_fabricantes{$fabricante}) {
$map_fabricantes{$fabricante} = $col_counter++;
}
return $map_fabricantes{$fabricante};
}

# =====
# OBTENER
# =====

sub obtener {
my ($self, $fila, $col) = @_;
my $cab_fila = $self->_buscar_cab_fila($fila);
return undef unless defined $cab_fila;

my $actual = $cab_fila->get_right();
while (defined $actual) {
return $actual if ($actual->get_col() == $col);
last if ($actual->get_col() > $col);
$actual = $actual->get_right();
}
return undef;
}

# =====
# IMPRESIÓN
# =====

sub mostrar {
my $self = shift;
print "\n=== MATRIZ DISPERSA (Proveedores vs Fabricantes) ===\n";
print "Total relaciones activas: $self->{total_datos}\n\n";

if (!defined $self->{lista_filas}) {
print "(matriz vacía)\n";
return;
}

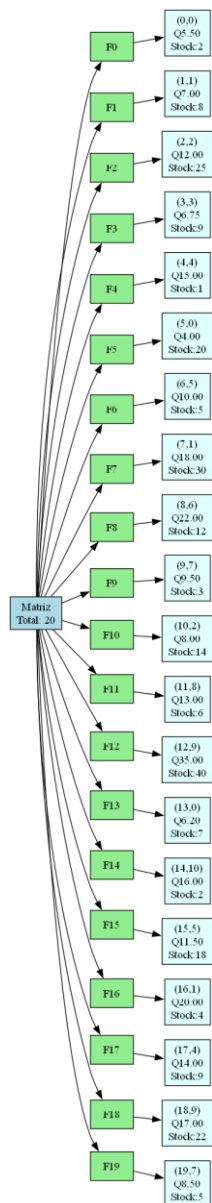
```



```
}

my $cab_fila = $self->{lista_filas};
while (defined $cab_fila) {
print "Proveedor (fila $cab_fila->{label}):\n";
my $nodo = $cab_fila->get_right();
while (defined $nodo) {
print " → Fabricante (col $nodo->{col}): Cantidad total = " . $nodo->get_valor() . "\n";
$nodo = $nodo->get_right();
}
$cab_fila = $cab_fila->get_next();
}
print "=====\n\n";
}

1;
```



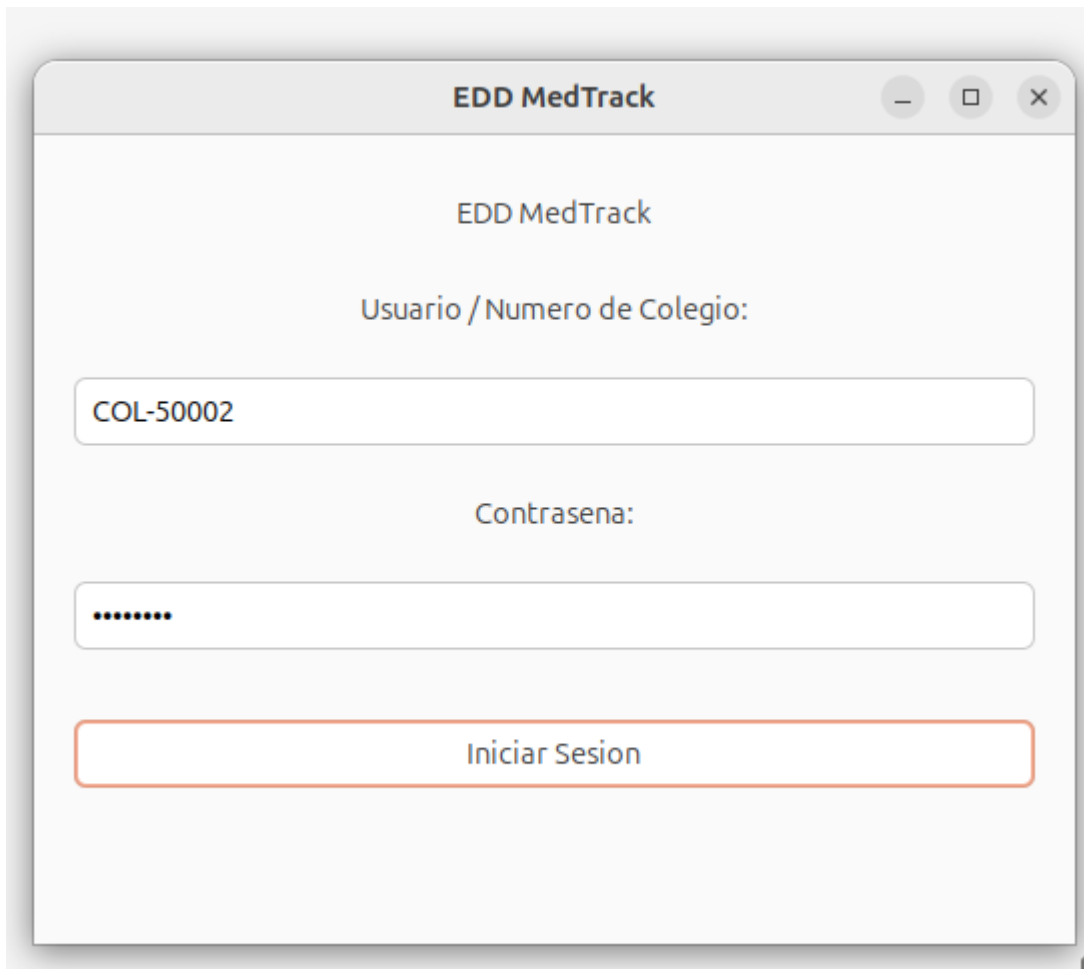
INTERFAZ GRÁFICA

La interfaz gráfica del sistema fue desarrollada utilizando GTK3.

El sistema cuenta con diferentes vistas, entre las cuales se incluyen:

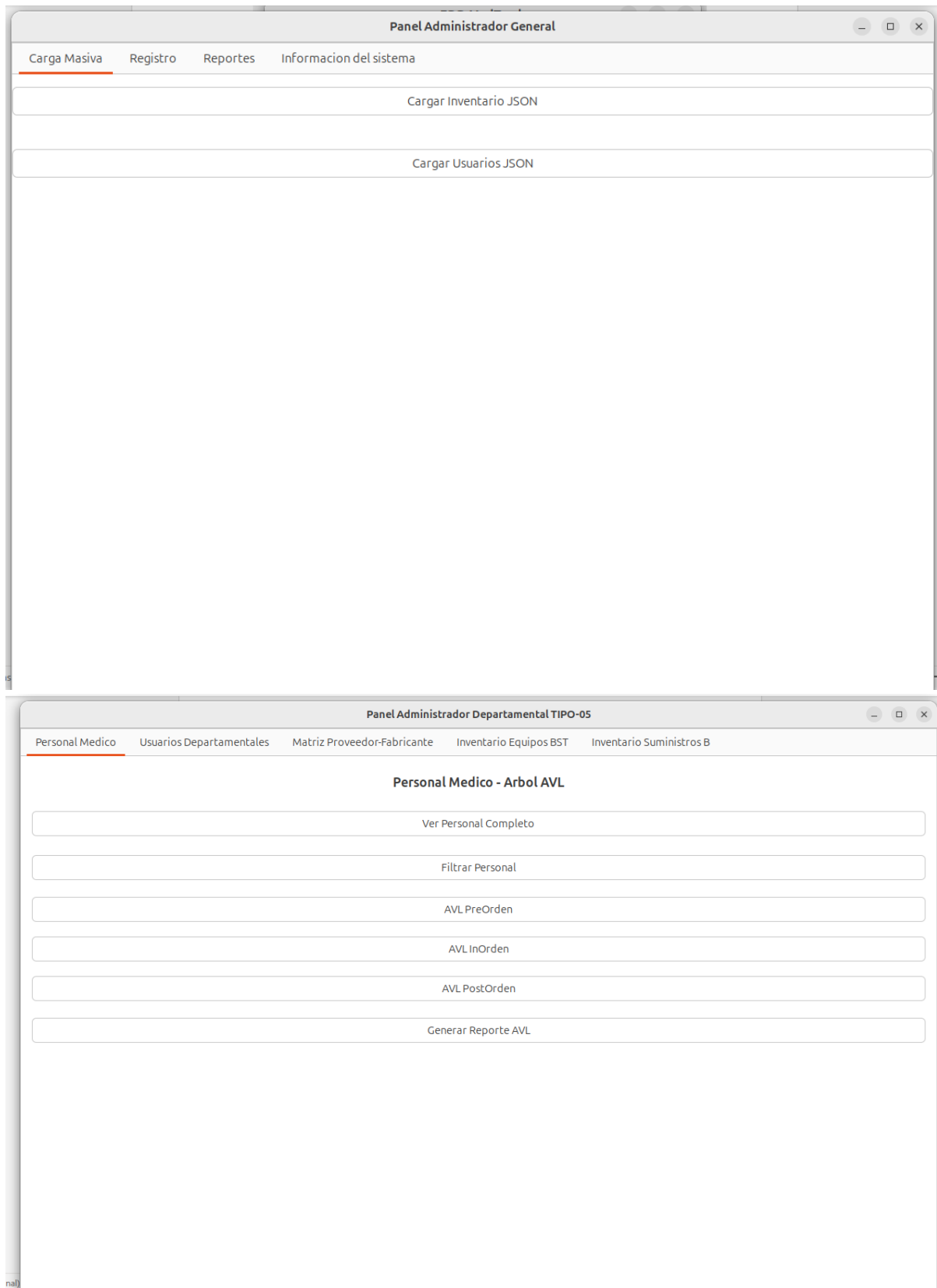
- Pantalla de inicio de sesión
- Panel de administrador general
- Panel de administrador tipo-05
- Panel de usuario normal
- Ventanas de visualización de datos

La interfaz permite interactuar de manera sencilla con las estructuras de datos.



The image shows a window titled "EDD MedTrack" with standard window controls (minimize, maximize, close). The window contains a login form with the following elements:

- The title "EDD MedTrack" is centered at the top of the window content.
- A label "Usuario / Numero de Colegio:" is positioned above the first input field.
- A text input field containing the value "COL-50002".
- A label "Contraseña:" is positioned above the second input field.
- A password input field represented by a series of dots ".....".
- A button labeled "Iniciar Sesion" with an orange border.



FUNCIONALIDADES PRINCIPALES

El sistema implementa diversas funcionalidades, entre las cuales destacan:

- Inicio de sesión con validación de credenciales
- Registro manual de datos
- Carga masiva mediante archivos JSON
- Búsqueda de información
- Eliminación de registros
- Visualización de datos mediante tablas
- Generación de reportes gráficos

GENERACIÓN DE REPORTES

El sistema permite generar reportes gráficos utilizando Graphviz.

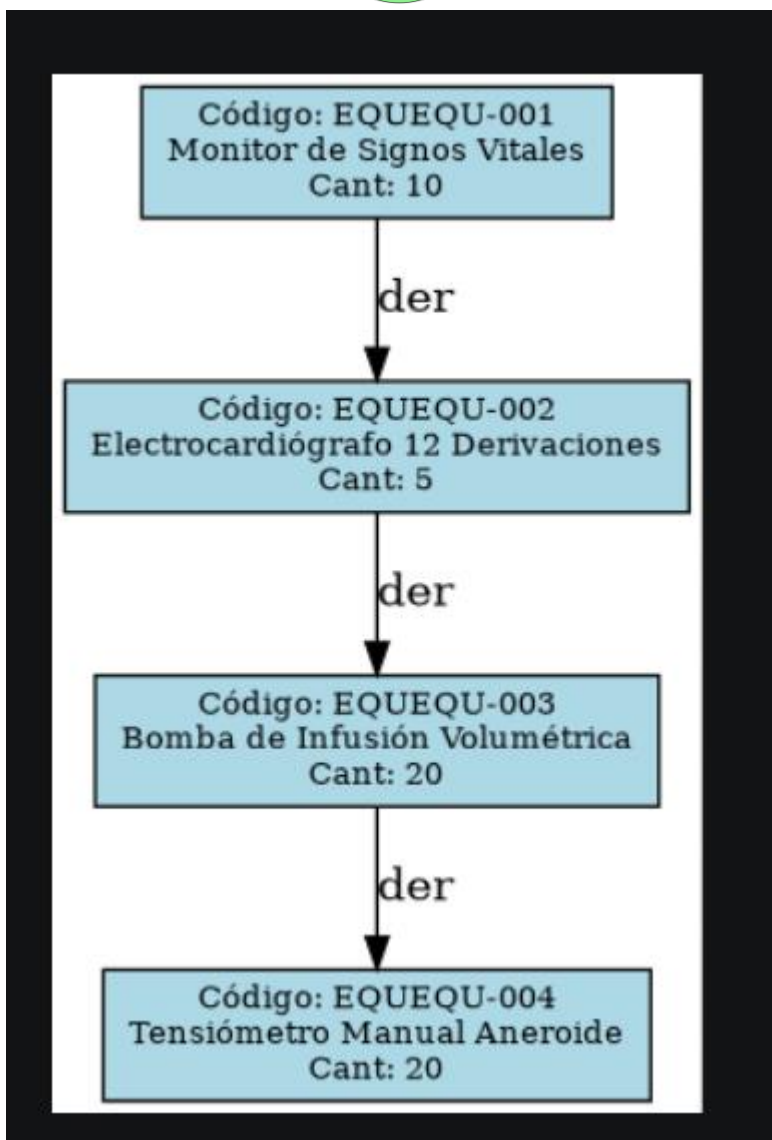
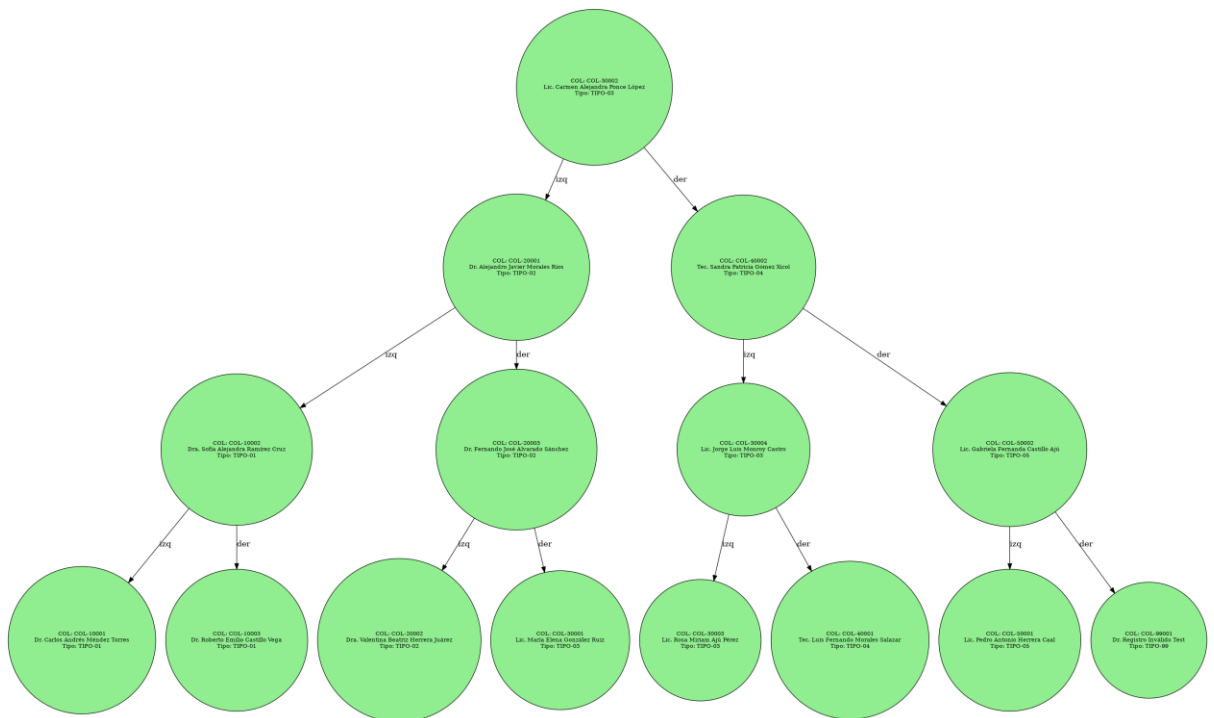
Estos reportes representan visualmente las estructuras de datos utilizadas, permitiendo comprender su organización interna.

Los reportes incluyen:

- Árbol AVL
- Árbol BST
- Árbol B

Además, estos reportes son mostrados directamente dentro de la interfaz gráfica del sistema.

 **AQUÍ INSERTAR IMÁGENES:**



```
{<f0> SUMSUM-001 | <f1> SUMSUM-002 | }
```

RECORRIDOS DE ESTRUCTURAS

Se implementaron distintos tipos de recorridos en las estructuras de datos:

- PreOrden
- InOrden
- PostOrden

El recorrido InOrden es especialmente útil ya que permite visualizar los datos de forma ordenada.

Panel Administrador Departamental TIPO-05

Personal Medico Usuarios Departamentales Matriz Proveedor-Fabricante Inventario Equipos BST Inventario Suministros B

Colegio	Nombre	Tipo	Especialidad	Departamento
COL-10001	Dr. Carlos Andrés Méndez Torres	TIPO-01	Medicina General	DEP-MED
COL-10002	Dra. Sofía Alejandra Ramírez Cruz	TIPO-01	Medicina Interna	DEP-MED
COL-10003	Dr. Roberto Emilio Castillo Vega	TIPO-01	Medicina Familiar	DEP-MED
COL-20001	Dr. Alejandro Javier Morales Ríos	TIPO-02	Cirugía General	DEP-CIR
COL-20002	Dra. Valentina Beatriz Herrera Juárez	TIPO-02	Cirugía Cardiovascular	DEP-CIR
COL-20003	Dr. Fernando José Alvarado Sánchez	TIPO-02	Cardiología	DEP-MED
COL-30001	Lic. María Elena González Ruiz	TIPO-03	N/A	DEP-FAR
COL-30002	Lic. Carmen Alejandra Ponce López	TIPO-03	N/A	DEP-MED
COL-30003	Lic. Rosa Miriam Ajú Pérez	TIPO-03	N/A	DEP-CIR
COL-30004	Lic. Jorge Luis Monroy Castro	TIPO-03	N/A	DEP-FAR
COL-40001	Tec. Luis Fernando Morales Salazar	TIPO-04	Hematología Clínica	DEP-LAB
COL-40002	Tec. Sandra Patricia Gómez Xicol	TIPO-04	Microbiología	DEP-LAB
COL-50001	Lic. Pedro Antonio Herrera Caal	TIPO-05	Administración Hospitalaria	DEP-ADM
COL-50002	Lic. Gabriela Fernanda Castillo Ajú	TIPO-05	Gestión de Salud	DEP-ADM
COL-99001	Dr. Registro Inválido Test	TIPO-99	N/A	DEP-XYZ

CARGA MASIVA DE DATOS

El sistema permite la carga masiva de información mediante archivos en formato JSON.

Esto facilita la inserción de grandes cantidades de datos sin necesidad de ingresarlos manualmente.

```
#!/usr/bin/perl
use strict;
use warnings;
use lib '.';

use JSON::PP; # ← Solo importamos el módulo (sin qw)

use ListaDobleEnlazada;
use ArbolBST;
use ArbolB;
use ArbolAVL;
use ListaCircularDobleProveedores;
use MatrizDispersaProveedorFabricante;
use Medicamento;
use NodoEquipo;
use NodoSuministro;
use NodoPersonal;
use Proveedor;

package CargaJSON;

# =====
# CARGA MASIVA INVENTARIO
# =====

sub cargar_inventario {
    my ($archivo, $lista_medicamentos, $bst_equipos, $btree_suministros,
        $lista_proveedores, $matriz) = @_;

    open my $fh, '<', $archivo or die "No se pudo abrir $archivo: $!";
    my $json_text = do { local $/; <$fh> };
    close $fh;

    my $data = JSON::PP::decode_json($json_text); # ←←← CORRECCIÓN AQUÍ

    my $contador = 0;

    foreach my $prov_hash (@{$data->{proveedor}}) {
        my $nit = $prov_hash->{nit};
```



```
my $nombre = $prov_hash->{nombre};
my $telefono = $prov_hash->{telefono};
my $direccion = $prov_hash->{direccion};

my $proveedor = $lista_proveedores->buscar_proveedor($nit);
if (!defined $proveedor) {
    $proveedor = Proveedor->new($nit, $nombre, $telefono, $direccion);
    $lista_proveedores->insertar_proveedor($proveedor);
}

foreach my $item (@{$prov_hash->{entrega}}) {
    my $tipo = $item->{tipo};
    $contador++;

    if ($tipo eq "MEDICAMENTO") {
        my $med = Medicamento->new(
            $item->{codigo},
            $item->{nombre},
            $item->{principio_activo} // "N/A",
            $item->{fabricante},
            $item->{precio_unitario},
            $item->{cantidad},
            $item->{fecha_vencimiento},
            $item->{nivel_minimo}
        );
        $lista_medicamentos->insertar_ordenado($med);

    } elsif ($tipo eq "EQUIPO") {
        my $equipo = NodoEquipo->new(
            "EQU" . $item->{codigo},
            $item->{nombre},
            $item->{fabricante},
            $item->{precio_unitario},
            $item->{cantidad},
            $item->{fecha_ingreso} // "2026-01-01",
            $item->{nivel_minimo}
        );
        $bst_equipos->insertar($equipo);

    } elsif ($tipo eq "SUMINISTRO") {
        my $sum = NodoSuministro->new(
            "SUM" . $item->{codigo},
            $item->{nombre},
            $item->{fabricante},
            $item->{precio_unitario},
            $item->{cantidad},
```

```

$item->{fecha_vencimiento} // "2028-12-31",
$item->{nivel_minimo}
);
$btree_suministros->insertar($sum);
}

$matriz->insertar($nit, $item->{fabricante}, $item->{cantidad});

$proveedor->agregar_entrega(
$prov_hash->{fecha_entrega},
$prov_hash->{numero_factura},
$tipo,
$item->{codigo},
$item->{nombre},
$item->{fabricante},
$item->{precio_unitario},
$item->{cantidad}
);
}
}

print "☑ Carga masiva de inventario completada: $contador productos procesados.\n";
}

# =====
# CARGA MASIVA USUARIOS
# =====

sub cargar_usuarios {
my ($archivo, $avl_personal) = @_ ;

open my $fh, '<', $archivo or die "No se pudo abrir $archivo: $!";
my $json_text = do { local $/; <$fh> };
close $fh;

my $data = JSON::PP::decode_json($json_text);

my $cont = 0;
foreach my $u (@{$data->{usuarios}}) {
my $usuario = NodoPersonal->new(
$u->{numero_colegio},
$u->{nombre_completo},
$u->{tipo_usuario},
$u->{departamento},
$u->{especialidad} // "N/A",
$u->{contrasena}
);

```

```

$avl_personal->insertar($usuario);
$cont++;
}

print "☑ Carga masiva de usuarios completada: $cont usuarios insertados en AVL.\n";
}

1;

```

```

1  #!/usr/bin/perl
2  use strict;
3  use warnings;
4  use lib '.';
5
6  use JSON::PP;          # ← Solo importamos el módulo (sin qw)
7
8  use ListaDobleEnlazada;
9  use ArbolBST;
10 use ArbolB;
11 use ArbolAVL;
12 use ListaCircularDobleProveedores;
13 use MatrizDispersaProveedorFabricante;
14 use Medicamento;
15 use NodoEquipo;
16 use NodoSuministro;
17 use NodoPersonal;
18 use Proveedor;
19
20 package CargaJSON;
21
22 # =====
23 # CARGA MASIVA INVENTARIO
24 # =====
25 sub cargar_inventario {
26     my ($archivo, $lista_medicamentos, $bst_equipos, $btree_suministros,
27         $lista_proveedores, $matriz) = @_;
28
29     open my $fh, '<', $archivo or die "No se pudo abrir $archivo: $!";
30     my $json_text = do { local $/; <$fh> };
31     close $fh;
32
33     my $data = JSON::PP::decode_json($json_text); # ←←← CORRECCIÓN AQUÍ
34
35     my $contador = 0;
36
37     foreach my $prov_hash (@{$data->{proveedor}}) {
38         my $nit

```

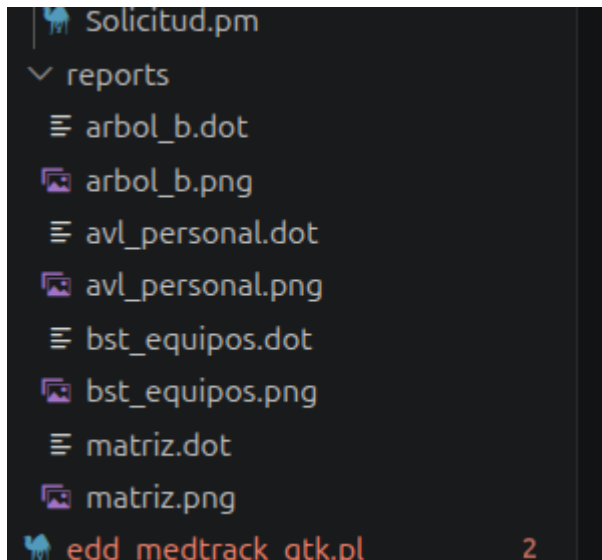
```
2 proveedor : {
3   {
10     "entrega": [
55     {
63       "fecha_vencimiento": "2027-02-14",
64       "nivel_minimo": 300
65     },
66     {
67       "tipo": "EQUIPO",
68       "codigo": "EQU-001",
69       "nombre": "Monitor de Signos Vitales",
70       "fabricante": "MediTechGT",
71       "precio_unitario": 4500.00,
72       "cantidad": 10,
73       "fecha_ingreso": "2026-03-10",
74       "nivel_minimo": 2
75     },
76     {
77       "tipo": "EQUIPO",
78       "codigo": "EQU-002",
79       "nombre": "Electrocardiógrafo 12 Derivaciones",
80       "fabricante": "MediTechGT",
81       "precio_unitario": 6200.00,
82       "cantidad": 5,
83       "fecha_ingreso": "2026-03-10",
84       "nivel_minimo": 1
85     },
86     {
87       "tipo": "EQUIPO",
88       "codigo": "EQU-003",
89       "nombre": "Bomba de Infusión Volumétrica",
90       "fabricante": "BuenaVidaGT",
91       "precio_unitario": 1800.00,
92       "cantidad": 20,
93       "fecha_ingreso": "2026-03-10",
94       "nivel_minimo": 5
95     },
96     {
97       "tipo": "SUMINISTRO",
98       "codigo": "SIM-001"
```

PRUEBAS DEL SISTEMA

Se realizaron diversas pruebas para verificar el correcto funcionamiento del sistema, incluyendo:

- Inserción de datos
- Eliminación
- Búsqueda
- Visualización

Los resultados obtenidos fueron satisfactorios, ya que todas las operaciones se ejecutaron correctamente.



PROBLEMAS ENCONTRADOS

Durante el desarrollo del sistema se presentaron algunas dificultades, entre ellas:

- Integración de GTK3 con Perl
- Manejo de estructuras complejas
- Visualización de reportes

Estas dificultades fueron solucionadas mediante pruebas constantes y depuración del código.

ANÁLISIS DE COMPLEJIDAD

Las estructuras implementadas presentan las siguientes complejidades:

- Árbol AVL: $O(\log n)$
- Árbol BST: $O(\log n)$ en promedio
- Árbol B: $O(\log n)$

Esto permite un manejo eficiente de la información.

CONCLUSIONES

El desarrollo de este proyecto permitió aplicar de manera práctica los conceptos de estructuras de datos.

Se logró implementar un sistema funcional que integra múltiples estructuras, demostrando la importancia de elegir la estructura adecuada según el tipo de problema.